Git and Ansible in the Splunk Sandbox: Tutorial Manual

Oct 25, 2026 · @Austo

Start here

This manual teaches Git and Ansible from zero, using the Splunk sandbox as a safe practice lab. Nothing you type touches a real computer, so you can break things freely and reset.

Who it's for. Complete beginners start at Level 1 and need no Splunk, Git or Linux experience. Experienced engineers can skip to Level 3 or the Level 4 challenges.

Level	Who	What you do	Help available
1 Beginner	Never used Git or Ansible	Follow exact commands, one step at a time	Every step explained, hints, auto-check
2 Intermediate	Comfortable with Level 1	Steps tell you the goal, you choose the commands	Hints on request
3 Advanced	Working engineers	Realistic tasks with traps	One hint, then the answer
4 Expert	Practicing for real work or interviews	Open-ended challenge, success criteria only	None, scored

Opening the lab

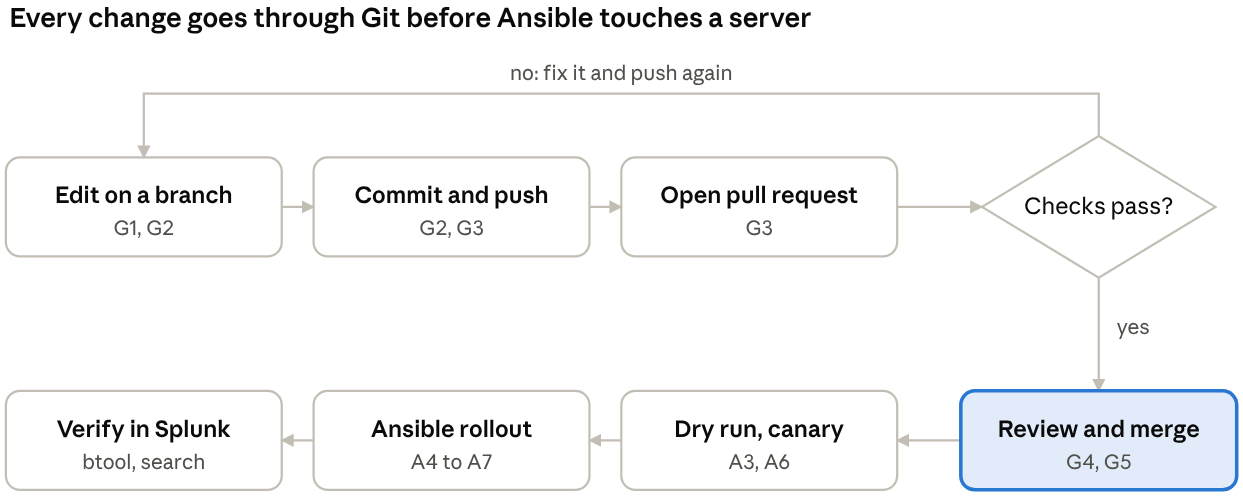
1. Start the sandbox and open it in your browser.
2. Go to **Apps → Engineer Sandbox → Learn**. Pick your name (or create a learner) so your progress is saved separately from anyone else's.
3. Choose a lesson. It opens in the **Lab** environment, a small practice company with 3 indexers and 50 forwarders, so screens stay simple.
4. The lesson panel sits on the right. The terminal and the **Code** app (the practice GitHub) open on the left.

Safety net

- **Reset lesson** puts everything back to how the lesson started.
- **Hint** gives a nudge. Asking again gives a bigger nudge, then the answer.
- **Explain** (or typing `explain` before any command, like `explain git commit`) describes what a command does without running it.

Each lesson in this manual has an ID like **G1** or **A3**. The same ID appears in the app.

The big picture



change workflow · Git first, then Ansible

A change starts on a branch, gets reviewed and tested in a pull request, and only after it merges does Ansible roll it out, one host first. The small labels show which lessons teach each step.

The ideas in plain English

Splunk in one minute

Splunk collects log data from thousands of computers and lets people search it. Small programs called **forwarders** run on each computer and send logs to **indexers**, which store them. People search from **search heads**. Splunk's behavior is controlled by text files ending in `.conf`, such as `inputs.conf` (what to collect) and `props.conf` (how to read it). Most of a Splunk engineer's job is changing those files safely on many servers at once. Git and Ansible are the tools for that.

Git: a history of every change

Git records every version of a set of files, who changed what, and why.

- **Repository (repo):** a project folder that Git tracks.
- **Commit:** a saved snapshot of your changes with a message explaining them.
- **Branch:** a separate line of work, so you can experiment without touching the main version.
- **main:** the official branch. What's on main is what gets deployed.
- **Push / pull:** send your commits to the shared server, or get other people's.
- **Pull request (PR):** a request to merge your branch into main. Teammates review it and automated checks test it first.
- **Merge:** combine a branch into main.

Ansible: one command, many servers

Ansible logs into servers and makes them match a description you write.

- **Control node:** the computer you run Ansible from (here, `ansible01`).
- **Inventory:** the list of servers, sorted into groups like `indexers` or `forwarders`.
- **Playbook:** a YAML file listing what should be true on which servers.
- **Task:** one step in a playbook, such as "copy this file" or "restart Splunk".
- **Module:** the tool a task uses, like `copy`, `template` or `service`.
- **Handler:** a task that runs only when something changed, typically a restart.
- **Idempotent:** running the same playbook twice changes nothing the second time. Ansible reports `ok` instead of `changed`.
- **Check mode (`--check`):** a dry run that shows what would change without changing it.

Level 1: Beginner

Type the commands exactly as shown. After each lesson the app checks your work and marks it done.

G1. Look around a repository (10 min)

Goal: see what a Git repo is and read its history.

1. In the terminal, log in to the automation server: `ssh svc_ansible@ansible01`

2. Go into the repo: `cd ~/splunk-automation`
3. List the files: `ls`
4. Ask Git what's going on: `git status`. It should say you're on branch `main` with nothing to commit.
5. See the history: `git log --oneline`. Each line is one commit: a short ID and a message.
6. Open the **Code** app and find the same repo and the same commits in the browser.

Check yourself: who made the most recent commit, and what did it change? (Hint: `git show`.)

G2. Make your first commit (15 min)

Goal: change a file and save that change in Git's history.

1. Open the readme in the editor: `nano README.md`
2. Add a line at the bottom, such as `Practice edit by <your name>`. Save with Ctrl+O, Enter, then exit with Ctrl+X.
3. See what changed: `git diff`. Added lines start with `+`.
4. Stage the change (tell Git you want it in the next commit): `git add README.md`
5. Run `git status` again. The file is now listed under "Changes to be committed".
6. Commit it: `git commit -m "Add practice line to README"`
7. Run `git log --oneline`. Your commit is at the top.

Check yourself: what's the difference between a changed file and a staged file?

G3. Branch, push and open a pull request (20 min)

Goal: propose a change the safe way, on a branch, for review.

1. Create and switch to a new branch: `git switch -c add-retention-note`
2. Edit `group_vars/indexers.yml` with nano and add a comment line starting with `#` that says what the file is for. Save and exit.
3. Stage and commit: `git add group_vars/indexers.yml` then `git commit -m "Document indexer variables"`
4. Send the branch to the shared server: `git push -u origin add-retention-note`
5. In the Code app, click **New pull request**, choose your branch, write a one-line description and create it.

6. Watch the checks run. When they're green and a teammate approves, click **Merge**.

Check yourself: why not commit straight to `main`?

A1. Read the inventory (10 min)

Goal: see which servers Ansible knows about.

1. Still on ansible01, in `~/splunk-automation`, run: `ansible-inventory --graph`
2. You'll see groups such as `indexers`, `search_heads`, `deployment_server` and `universal_forwarders`, with server names under each.
3. Open the file it came from: `cat inventories/lab/hosts.yml`

Check yourself: how many indexers are in the lab? Which group would you target to change every forwarder?

A2. Say hello to your servers (10 min)

Goal: run your first Ansible command.

1. Ping the indexers: `ansible indexers -m ping`. Each one should answer `SUCCESS` and `"ping": "pong"`. (This isn't a network ping; it checks Ansible can log in and run.)
2. Now try everyone: `ansible all -m ping`
3. One forwarder answers `UNREACHABLE!`. That's on purpose: it's switched off. Real fleets always have a few.

Check yourself: what does `-m` stand for?

A3. Your first dry run (15 min)

Goal: see what a playbook would change, without changing anything.

1. Open the playbook and read it: `cat playbooks/deploy_apps.yml`. Find the `hosts:` line and the list of `tasks:`.
2. Do a dry run on one server only: `ansible-playbook playbooks/deploy_apps.yml --check --diff --limit idx01`
3. Read the output. Each task shows `ok` (already correct), `changed` (would change) or `skipping`. `--diff` shows the exact lines that would change.
4. The last block, `PLAY RECAP`, totals it up.

Check yourself: what do `--check`, `--diff` and `--limit` each do? Always use all three before a real run on production.

Level 2: Intermediate

Each lesson gives a goal and the key commands. You work out the order. Use **Hint** if you're stuck for more than five minutes.

G4. Fix a pull request that fails its checks (20 min)

Situation: a teammate's PR in the `splunk-apps` repo adds a new log source, but the `btool-check` check is red.

- Open the PR in the Code app and read the failed check's log. It names the file and line.
- Get the branch locally: `cd ~/splunk-apps`, then `git fetch` and `git switch <branch-name>`.
- Fix the file, commit, and `git push`. The checks rerun automatically.

Done when: all checks are green and the PR is merged.

Learn: automated checks catch mistakes before they reach servers. Read the log before guessing.

G5. Stay in sync with your team (10 min)

Situation: someone merged changes to `main` while you were away.

- `git switch main`, then `git pull`.
- Compare: `git log --oneline -5` before and after.
- Start new work from the fresh `main` with `git switch -c <new-branch>`.

Done when: your local `main` matches the server.

A4. Variables and templates (20 min)

Situation: indexes need a bigger size limit. The value lives in a variable, and a template turns it into `indexes.conf`.

- Find the variable: `grep -r max_total_data_size group_vars/`
- Find where it's used: `cat roles/splunk_indexes/templates/indexes.conf.j2`. The `{{ ... }}` parts are filled in from variables.
- Change the value on a branch, then dry-run: `ansible-playbook playbooks/cluster_bundle.yml --check --diff`
- Open a PR, merge it, then run the playbook for real.

Done when: `splunk btool indexes list --debug` on an indexer shows the new value.

Learn: change variables, not the generated files. The template keeps every server consistent.

A5. Handlers and idempotency (15 min)

Situation: Splunk only needs a restart when a config file actually changed.

- Open `roles/splunk_outputs/tasks/main.yml` and find `notify: restart splunk`.
- Open `roles/splunk_outputs/handlers/main.yml` to see the handler.
- Run `ansible-playbook playbooks/deploy_apps.yml --limit idx01` twice.

Done when: you can explain why the first run shows `changed` and a `RUNNING HANDLER` line, and the second run shows only `ok`.

Learn: a well-written playbook is safe to run again. If the second run shows `changed`, something is wrong.

A6. Canary first, then everyone (20 min)

Situation: a new `outputs.conf` needs to go to all 50 forwarders.

- Pick one forwarder as the canary and run the playbook with `--limit` on it only.
- Confirm it still sends data: in Search, `index=_internal host=<canary> earliest=-5m | stats count`
- Then run for the whole `universal_forwarders` group.

Done when: all forwarders run the new config and none went quiet.

Learn: one host first, check it, then the rest. This is the habit interviewers look for.

Level 3: Advanced

Realistic tasks with a trap in each. You get a goal and success criteria, one hint, then the answer. Level 3 runs on the Lab or APL environment.

G6. Resolve a merge conflict (20 min)

Situation: your PR changes the `fortinet` stanza in `indexes.conf`. A teammate merged a change to the same stanza first, and now your PR says it can't merge.

- Update your branch: `git switch <your-branch>`, then `git pull origin main`.

- Git marks the conflict in the file with `<<<<<<<`, `=======` and `>>>>>>>`. Edit the file to keep both changes and remove the markers.
- `git add` the file, `git commit`, `git push`.

Trap: deleting the teammate's lines to make the conflict go away.

Done when: both changes are in `main` and checks are green.

G7. Undo a bad change safely (15 min)

Situation: a merged commit broke parsing for one sourcetype. Roll it back without rewriting history.

- Find it: `git log --oneline` and `git show <commit-id>`.
- Undo it with a new commit: `git revert <commit-id>`, then push and open a PR.

Trap: `git reset --hard` followed by `git push --force` on `main`. The sandbox blocks force-pushes to `main` and records the attempt.

Done when: the revert is merged and deployed, and events parse correctly again.

A7. Rolling restart without an outage (30 min)

Situation: `playbooks/rolling_restart.yml` restarts every indexer at once. Last time it took search down for 20 minutes. Fix it.

- Restart one indexer at a time with `serial: 1`.
- Put the cluster in maintenance mode first, from the cluster manager (`splunk enable maintenance-mode`), and turn it off at the end (`splunk disable maintenance-mode`). Use `run_once` and `delegate_to` for those tasks.
- Wait for each peer to come back before moving on (`wait_for` on port 8089, or check cluster status).
- Dry-run with `--check`, then run it.

Trap: forgetting to disable maintenance mode afterwards, which leaves bucket fixup paused.

Done when: every peer restarted, search stayed up, and the search and replication factors are met.

A8. Secrets with Ansible Vault (20 min)

Situation: the forwarder certificate password is changing.

- See encrypted secrets: `ansible-vault view group_vars/all/vault.yml --vault-password-file ~/.vault_pass`
- Edit with `ansible-vault edit` (same options) and update the password.
- Find every place the old secret is referenced: `grep -r vault_ group_vars/ host_vars/`
- Run `playbooks/cert_rotate.yml` on a canary forwarder, then the rest.

Trap: a second copy of the secret in a host-specific vars file keeps the old value.

Done when: all forwarders reconnect over TLS and no plaintext secret was committed.

A9. The servers Ansible forgot (20 min)

Situation: two new indexers joined the cluster last week, but they never got the latest config.

- Compare what Ansible knows (`ansible-inventory --graph`) with what the cluster has (Settings → Indexer clustering, or `splunk show cluster-status` on the cluster manager).
- Add the missing hosts to the inventory on a branch, open a PR, merge, and run the playbook with `--limit` on the new hosts.

Done when: `btool` output matches on every indexer.

A10. The fix that kept disappearing (25 min)

Situation: someone fixed `props.conf` directly on a server. Every night it reverts.

- Work out what's overwriting it (the nightly Ansible run from the repo).
- Put the fix in the repo, through a PR, so Ansible deploys it instead of undoing it.

Done when: after the next nightly run the fix is still there.

Learn: with automation, the repo is the source of truth. Manual edits on servers get erased.

Level 4: Expert challenges

No steps and no hints. You get a ticket, a time limit and success criteria. The app scores how you worked (safe habits, checks run, canaries used) as well as whether it ended up correct. These run on the APL environment.

ID	Challenge	Success criteria	Time
X1	Write a new role, `splunk_hec`, that enables an HTTP Event Collector token on the heavy forwarders	Role has defaults, tasks, a template and a handler; passes `ansible-lint`; second run reports zero changes; merged through a PR	45 min
X2	Upgrade every universal forwarder, site by site, without losing more than 2% of hosts at once	Uses `serial` batches and `max_fail_percentage`; canary batch first; play stops automatically if a batch fails; no site goes quiet	45 min
X3	Make dry runs mandatory	Add a CI check to `splunk-automation` that runs `--check --diff` against the lab inventory and posts the diff to the PR; a PR with an unexpected change fails	40 min
X4	A playbook fails on 6 of 20 indexers with a vague error	Find the root cause from `-vvv` output and host facts, fix it in code, rerun only on the failed hosts (`--limit @playbooks/rolling_restart.retry` or a host list)	30 min
X5	Production incident: a merged `props.conf` change broke FortiGate parsing during peak hours	Find the bad commit, revert through a PR, redeploy the bundle, confirm events parse, write a five-line postmortem in the ticket	40 min
X6	Servers were changed by hand for months; bring them back under code	`--check` against all hosts shows zero changes at the end; every real difference is captured in the repo, not overwritten	60 min

Scoring penalties: running against all hosts without a dry run or canary, force-pushing to `main`, merging with failing checks, committing a plaintext secret, and leaving maintenance mode enabled.

Cheat sheets

Git

Command	What it does
`git status`	Shows your branch and which files changed or are staged
`git log --oneline`	Lists commits, newest first
`git show <id>`	Shows one commit's message and changes
`git diff`	Shows unstaged changes (`git diff --staged` for staged)
`git add <file>`	Stages a file for the next commit
`git commit -m "message"`	Saves staged changes as a commit
`git switch -c <branch>`	Creates a branch and moves to it
`git switch <branch>`	Moves to an existing branch
`git pull`	Gets and merges the latest commits from the server
`git push -u origin <branch>`	Sends a new branch to the server (later just `git push`)
`git fetch`	Downloads server changes without merging them
`git stash` / `git stash pop`	Sets uncommitted changes aside, then brings them back
`git revert <id>`	Undoes a commit with a new commit (safe on shared branches)
`git blame <file>`	Shows who last changed each line
`git restore <file>`	Throws away uncommitted changes to a file

Ansible

Command	What it does
`ansible-inventory --graph`	Shows groups and hosts
`ansible <group> -m ping`	Checks Ansible can reach and run on hosts
`ansible <group> -m command -a "splunk status"`	Runs one command on a group (ad hoc)
`ansible-playbook <file> --syntax-check`	Checks the YAML parses, runs nothing
`ansible-playbook <file> --check --diff`	Dry run showing exact changes
`ansible-playbook <file> --limit <host>`	Runs on one host or group only
`ansible-playbook <file> --tags <tag>`	Runs only tasks with that tag
`ansible-playbook <file> -e var=value`	Overrides a variable for this run
`ansible-playbook <file> -v` (to `-vvv`)	More detail, for debugging
`ansible-lint <file>`	Flags bad practice and risky patterns
`ansible-vault view/edit <file>`	Reads or edits an encrypted secrets file

Playbook keywords

Keyword	Meaning
`hosts:`	Which inventory group the play targets
`become: true`	Run as an administrator (sudo)
`tasks:`	The ordered list of steps
`notify:` / `handlers:`	Trigger a step (like a restart) only when something changed
`when:`	Run a task only if a condition is true
`serial:`	Work through hosts in batches (1, 2, or a percentage)

Keyword	Meaning
`max_fail_percentage:`	Stop the play if too many hosts in a batch fail
`run_once:` / `delegate_to:`	Run a task once, on a specific host (for example the cluster manager)
`register:`	Save a task's result in a variable
`{{ variable }}`	Insert a variable's value (Jinja2)

When you're stuck

First read the error's last line, which usually names the problem. Then check the table. If you're still stuck: **Hint**, then **Reset lesson**. Resetting is normal, not failure.

Git messages

You see	It means	Do this
`fatal: not a git repository`	You're not inside a repo folder	`cd ~/splunk-automation` (or `~/splunk-apps`)
`nothing to commit, working tree clean`	No changes since the last commit	Edit a file first, or you're already done
`Please tell me who you are`	Git doesn't know your name yet	`git config --global user.name "Your Name"` and `git config --global user.email you@example.com`
`Updates were rejected because the remote contains work that you do not have locally`	Someone pushed first	`git pull`, fix any conflict, then `git push`
`CONFLICT (content): Merge conflict in <file>`	Two people changed the same lines	See lesson G6
`Your branch is ahead of 'origin/main' by 1 commit`	Your commit isn't on the server yet	`git push` (on a branch, through a PR)

You see	It means	Do this
`force-push to main is blocked`	The sandbox protects `main`	Use `git revert` and a PR instead

Ansible messages

You see	It means	Do this
`UNREACHABLE! ... Failed to connect to the host via ssh`	The host is down or offline	Check it in Fleet Overview; leave it out with `--limit '!hostname'` if it's expected
`Syntax Error while loading YAML`	Bad indentation or a tab character	YAML uses spaces only; line up items under their parent
`The task includes an option with an undefined variable`	A `{{ variable }}` has no value	Find where it should be set (`grep -r <name> group_vars host_vars roles`)
`couldn't resolve module/action`	Module name typo, or not supported	Check spelling against the module list
`Missing sudo password`	The task needs admin rights	Add `become: true` and run with `-K` if prompted
`Attempting to decrypt but no vault secrets found`	The play uses a vaulted file	Add `--vault-password-file ~/.vault_pass`
`Could not match supplied host pattern`	Typo in a group or host name	Compare with `ansible-inventory --graph`
A task shows `changed` on every run	The task isn't idempotent	Use a module like `template` or `lineinfile` instead of `command`

Habits that prevent most problems

1. Run `git status` before and after everything.
2. Never work directly on `main`.
3. Run `--check --diff` before every real playbook run.
4. Try one host with `--limit` before many.

Glossary

Term	Meaning
Ad hoc command	A one-off Ansible command run without a playbook
Branch	A separate line of work in Git
btool	Splunk's tool for showing the final, merged value of every config setting and which file it came from
Bundle push	Sending a set of configs from a central server (deployer or cluster manager) to many Splunk servers
Canary	One host changed first to test a change before the rest
CI checks	Automated tests that run on every pull request
Cluster manager	The Splunk server that coordinates the indexers
Commit	A saved snapshot of changes in Git
Deployer	The server that pushes apps to the search head cluster
Deployment server	The server that pushes configs to forwarders
Forwarder	Small Splunk agent that sends logs from a computer
Handler	An Ansible task that runs only when notified by a change
Idempotent	Safe to run again; the second run changes nothing
Indexer	Splunk server that stores data and answers searches
Inventory	Ansible's list of hosts and groups
Jinja2	The template language Ansible uses for `{{ variables }}`
main	The official Git branch that gets deployed

Term	Meaning
Maintenance mode	A cluster setting that pauses data rebalancing during planned restarts
Merge conflict	Git can't combine two changes to the same lines automatically
Module	The tool behind an Ansible task (`copy` , `template` , `service`)
Playbook	A YAML file of plays and tasks for Ansible
Pull request	A request to merge a branch, with review and checks
Repository	A folder of files tracked by Git
Revert	Undo a commit by adding a new commit
Role	A reusable bundle of Ansible tasks, templates, handlers and defaults
Search head	Splunk server people search from
Vault	Ansible's way of encrypting secrets in files
YAML	The plain-text format playbooks are written in; indentation matters